

neixx/common/include/neixx/common/location.h

```
#pragma once

#ifndef NEI_COMMON_LOCATION_H
#define NEI_COMMON_LOCATION_H

#include <cstdint>

#include <nei/macros/nei_export.h>

namespace nei {

class NEI_API Location final {
public:
    constexpr Location() noexcept = default;

    constexpr Location(const char *file_name, std::int32_t line, const char *function_name)
noexcept
        : file_name_(file_name)
        , function_name_(function_name)
        , line_(line) {}

    static constexpr Location Current(const char *file_name, std::int32_t line, const char
*function_name) noexcept {
        return Location(file_name, line, function_name);
    }

    static constexpr Location Unknown() noexcept {
        return Location();
    }

    constexpr const char *file_name() const noexcept {
        return file_name_;
    }

    constexpr const char *function_name() const noexcept {
        return function_name_;
    }

    constexpr std::int32_t line() const noexcept {
        return line_;
    }

    constexpr bool is_null() const noexcept {
        return file_name_ == nullptr;
    }

private:
    const char *file_name_ = nullptr;
    const char *function_name_ = nullptr;
};

}
```

```

    std::int32_t line_ = 0;
    std::int32_t reserved_ = 0;
};

static_assert(sizeof(Location) == sizeof(const char *) * 2 + sizeof(std::int32_t) * 2,
               "Location layout must stay fixed for ABI stability.");

} // namespace nei

#define FROM_HERE ::nei::Location::Current(__FILE__, __LINE__, __FUNCTION__)

#endif // NEI_COMMON_LOCATION_H

```

neixx/common/include/neixx/common/time_source.h

```

#pragma once

#ifndef NEI_COMMON_TIME_SOURCE_H
#define NEI_COMMON_TIME_SOURCE_H

#include <chrono>

#include <nei/macros/nei_export.h>

namespace nei {

class NEI_API TimeSource {
public:
    virtual ~TimeSource();

    virtual std::chrono::steady_clock::time_point Now() const = 0;
};

class NEI_API SystemTimeSource final : public TimeSource {
public:
    static const SystemTimeSource &Instance();

    std::chrono::steady_clock::time_point Now() const override;

private:
    SystemTimeSource() = default;
};

} // namespace nei

#endif // NEI_COMMON_TIME_SOURCE_H

```

neixx/common/src/time_source.cpp

```
#include <neixx/common/time_source.h>

namespace nei {

TimeSource::~TimeSource() = default;

const SystemTimeSource &SystemTimeSource::Instance() {
    static const SystemTimeSource instance;
    return instance;
}

std::chrono::steady_clock::time_point SystemTimeSource::Now() const {
    return std::chrono::steady_clock::now();
}

} // namespace nei
```

neixx/functional/include/neixx/functional/bind.h

```
#pragma once

#ifndef NEIXX_FUNCTIONAL_BIND_H_
#define NEIXX_FUNCTIONAL_BIND_H_

#include <tuple>
#include <type_traits>
#include <utility>

#include <neixx/functional/callback.h>

namespace nei {

// --- BindOnce -----
//
// Binds a callable and zero or more arguments into a move-only OnceCallback.
//
template <typename F, typename... Args>
OnceCallback BindOnce(F &&functor, Args &&...args) {
    using Fn = std::decay_t<F>;
    using BoundArgs = std::tuple<detail::bind_arg_storage_t<std::decay_t<Args>>...>;
    static_assert(std::is_invocable_v<Fn, detail::bind_arg_storage_t<std::decay_t<Args>>...>,
                  "BindOnce: functor is not callable with the provided argument types.");

    auto bound_lambda = [fn = Fn(std::forward<F>(functor)),
                        args = BoundArgs(detail::StoreBoundArg(std::forward<Args>(args))...)]
() mutable {
    // WeakPtr safety: if the first bound arg is a WeakPtr and has expired,
    // silently skip invocation - no external null-check required.
    if constexpr (sizeof...(Args) > 0) {
```

```

        if constexpr (detail::is_weak_ptr_v<std::decay_t<std::tuple_element_t<0, BoundArgs>>>)
        {
            if (!std::get<0>(args))
                return;
        }
    }
    std::apply([&](auto &...a) { std::invoke(std::move(fn), detail::UnwrapBoundArg(a)...); }, args);
};
return OnceCallback(std::move(bound_lambda));
}

// --- BindRepeating -----
//
// Binds a callable and zero or more arguments into a copyable RepeatingCallback.
// Copies share the same underlying allocation via reference counting.
//
template <typename F, typename... Args>
RepeatingCallback BindRepeating(F &&functor, Args &&...args) {
    using Fn = std::decay_t<F>;
    using BoundArgs = std::tuple<detail::bind_arg_storage_t<std::decay_t<Args>>...>;
    static_assert((!detail::is_passed_wrapper_v<std::decay_t<Args>> && ...),
        "BindRepeating: Passed() is not supported. Use BindOnce for move-only arguments.");
    static_assert(std::is_invocable_v<Fn &, detail::bind_arg_storage_t<std::decay_t<Args>> &...>,
        "BindRepeating: functor is not callable with lvalue references of bound arguments.");

    struct Storage {
        detail::RepeatingControlBlock ctrl; // MUST be first member
        Fn fn;
        BoundArgs bound;
    };

    auto *s = static_cast<Storage *>(detail::callback_alloc(sizeof(Storage)));
    s->ctrl.invoke = [](detail::RepeatingControlBlock *self) {
        auto *st = reinterpret_cast<Storage *>(self);
        // WeakPtr safety: if the first bound arg is a WeakPtr and has expired,
        // silently skip invocation - no external null-check required.
        if constexpr (sizeof...(Args) > 0) {
            if constexpr (detail::is_weak_ptr_v<std::decay_t<std::tuple_element_t<0, BoundArgs>>>)
            {
                if (!std::get<0>(st->bound))
                    return;
            }
        }
        std::apply([&](auto &...a) { std::invoke(st->fn, a...); }, st->bound);
    };
    s->ctrl.destroy = [](detail::RepeatingControlBlock *self) {
        if (self->ref_count.fetch_sub(1, std::memory_order_acq_rel) == 1) {
            auto *st = reinterpret_cast<Storage *>(self);
            st->fn.~Fn();
        }
    };
}

```

```

        st->bound.~BoundArgs();
        detail::callback_free(self);
    }
};
new (&s->ctrl.ref_count) std::atomic<int>(1);
new (&s->fn) Fn(std::forward<F>(functor));
new (&s->bound) BoundArgs(detail::StoreBoundArg(std::forward<Args>(args))...);
return RepeatingCallback(&s->ctrl);
}

} // namespace nei

#endif // NEIXX_FUNCTIONAL_BIND_H_

```

neixx/functional/include/neixx/functional/callback.h

```

#pragma once

#ifndef NEI_TASK_CALLBACK_H
#define NEI_TASK_CALLBACK_H

#include <atomic>
#include <functional>
#include <type_traits>
#include <utility>

#include <nei/macros/nei_export.h>
#include <neixx/functional/callback_base.h>
#include <neixx/functional/callback_internal.h>

namespace nei {

class OnceCallback;
class RepeatingCallback;

namespace detail {

template <typename F>
void InitOnceCallbackFromFunctor(OnceCallback &cb, F &&functor);

template <typename F>
void InitRepeatingCallbackFromFunctor(RepeatingCallback &cb, F &&functor);

// SBO buffer size for OnceCallback: 48 bytes allows most small lambdas and
// small bind objects to be stored inline without heap allocation.
constexpr std::size_t ONCE_SBO_SIZE = 48;
constexpr std::size_t ONCE_SBO_ALIGN = alignof(std::max_align_t);

// Function pointers for OnceCallback storage operations.
struct OnceCallbackvTable {
    // Invoke and destroy the functor in-place, then clean up storage.
    void (*invoke_and_destroy)(char *storage);
};

} // namespace detail

} // namespace nei

```

```

    // Destroy the functor in-place without invoking.
    void (*destroy)(char *storage);
};

// ABI-stable control block for RepeatingCallback.
// Shared-ownership semantics via an embedded reference count.
struct RepeatingControlBlock {
    void (*invoke)(RepeatingControlBlock *self); // run functor (non-consuming)
    void (*destroy)(RepeatingControlBlock *self); // decrement ref; free when count reaches 0
    std::atomic<int> ref_count;
};

// VTable for RepeatingCallback inline (SBO) storage path.
// copy_construct copies a functor from src into dst (dst has no prior state).
// destroy in-place destructs the functor without freeing the storage itself.
struct RepeatingInlineVTable {
    void (*invoke)(char *storage); // non-consuming invocation
    void (*copy_construct)(char *dst, const char *src); // copy-construct functor
    void (*destroy)(char *storage); // in-place destructor
};

// SBO parameters for RepeatingCallback - mirror OnceCallback for consistency.
constexpr std::size_t REPEATING_SBO_SIZE = ONCE_SBO_SIZE;
constexpr std::size_t REPEATING_SBO_ALIGN = ONCE_SBO_ALIGN;

} // namespace detail

#ifdef _MSC_VER
#pragma warning(push)
#pragma warning(disable : 4324)
#endif

// --- OnceCallback -----
//
// ABI-stable, move-only, single-shot callable wrapper (void() signature).
//
// Stable layout: fixed-size storage buffer with SBO (Small Buffer Optimization):
// - Storage: 48 bytes for inline functor + bound args (most small lambdas fit)
// - VTable: 2 function pointers (invoke_and_destroy, destroy) = 16 bytes
// - Padding: alignment to natural boundary
//
// Total public size: 64-72 bytes (depending on alignment), part of ABI contract.
// Small objects (<48 bytes) use inline storage (zero allocation).
// Large objects are heap-allocated via pointer in buffer.
//
// All non-template lifecycle methods are defined in callback.cpp and exported
// from the nei shared library, guaranteeing a single copy of each symbol.
//
class NEI_API OnceCallback : public CallbackBase {
public:
    OnceCallback() noexcept;
    ~OnceCallback();
    OnceCallback(OnceCallback &&) noexcept;

```

```

OnceCallback &operator=(OnceCallback &&) noexcept;

OnceCallback(const OnceCallback &) = delete;
OnceCallback &operator=(const OnceCallback &) = delete;

explicit operator bool() const noexcept;
void Run() &&;

// Implicit conversion from any void()-callable (no bound args).
// Instantiated in the caller's TU; stable lifecycle paths come from the DLL.
template <typename F, typename = std::enable_if_t<!std::is_same_v<std::decay_t<F>,
OnceCallback>>>
/*implicit*/ OnceCallback(F &&functor) {
    detail::InitOnceCallbackFromFunctor(*this, std::forward<F>(functor));
}

private:
    bool IsNullImpl() const noexcept override {
        return vtable_.invoke_and_destroy == nullptr;
    }

    detail::OnceCallbackVTable vtable_; // 16 bytes
    alignas(detail::ONCE_SBO_ALIGN) char storage_[detail::ONCE_SBO_SIZE]; // 48 bytes

    template <typename F>
    friend void detail::InitOnceCallbackFromFunctor(OnceCallback &cb, F &&functor);

    friend class RepeatingCallback;
};

// --- RepeatingCallback -----
//
// ABI-stable, copyable, multi-shot callable wrapper (void() signature).
// Copies share ownership via an embedded reference count in the control block.
//
class NEI_API RepeatingCallback : public CallbackBase {
public:
    RepeatingCallback() noexcept;
    ~RepeatingCallback();
    RepeatingCallback(const RepeatingCallback &) noexcept;
    RepeatingCallback &operator=(const RepeatingCallback &) noexcept;
    RepeatingCallback(RepeatingCallback &&) noexcept;
    RepeatingCallback &operator=(RepeatingCallback &&) noexcept;

    explicit operator bool() const noexcept;
    void Run() const;

    // Implicit conversion from any void()-callable (no bound args).
    template <typename F,
        typename = std::enable_if_t<!std::is_same_v<std::decay_t<F>, RepeatingCallback>
        && !std::is_same_v<std::decay_t<F>, OnceCallback>>>
    /*implicit*/ RepeatingCallback(F &&functor) {
        detail::InitRepeatingCallbackFromFunctor(*this, std::forward<F>(functor));
    }

```

```

}

// Internal: takes ownership of a pre-allocated control block (heap-only).
explicit RepeatingCallback(detail::RepeatingControlBlock *ctrl) noexcept;

private:
    bool IsNullImpl() const noexcept override {
        return inline_vtable_.invoke == nullptr && ctrl_ == nullptr;
    }

    // Inline path: inline_vtable_.invoke != nullptr; ctrl_ == nullptr.
    // Heap path: inline_vtable_ is zeroed; ctrl_ != nullptr.
    detail::RepeatingInlineVTable inline_vtable_{nullptr, nullptr, nullptr};
    // 24 bytes
    alignas(detail::REPEATING_SBO_ALIGN) mutable char
inline_storage_[detail::REPEATING_SBO_SIZE]; // 48 bytes
    detail::RepeatingControlBlock *ctrl_{nullptr};
    // 8 bytes

    template <typename F>
    friend void detail::InitRepeatingCallbackFromFunctor(RepeatingCallback &cb, F &&functor);
};

#ifdef _MSC_VER
#pragma warning(pop)
#endif

namespace detail {

template <typename F>
void InitOnceCallbackFromFunctor(OnceCallback &cb, F &&functor) {
    using Fn = std::decay_t<F>;
    if constexpr (is_sbo_eligible_v<Fn, ONCE_SBO_SIZE, ONCE_SBO_ALIGN>) {
        cb.vtable_.invoke_and_destroy = [](char *storage) {
            auto *fn = reinterpret_cast<Fn *>(storage);
            std::invoke(std::move(*fn));
            fn->~Fn();
        };
        cb.vtable_.destroy = [](char *storage) {
            auto *fn = reinterpret_cast<Fn *>(storage);
            fn->~Fn();
        };
        new (cb.storage_) Fn(std::forward<F>(functor));
    } else {
        struct HeapLayout {
            OnceCallbackVTable vt;
            Fn fn;
        };

        auto *h = static_cast<HeapLayout *>(callback_alloc(sizeof(HeapLayout)));
        h->vt.invoke_and_destroy = [](char *storage) {
            auto *ptr = *reinterpret_cast<HeapLayout **>(storage);
            std::invoke(std::move(ptr->fn));
        };
    }
}

```



```

        ptr->fn.~Fn();
        callback_free(ptr);
    };
    h->vt.destroy = [](char *storage) {
        auto *ptr = *reinterpret_cast<HeapLayout **>(storage);
        ptr->fn.~Fn();
        callback_free(ptr);
    };
    new (&h->fn) Fn(std::forward<F>(functor));
    *reinterpret_cast<HeapLayout **>(cb.storage_) = h;
    cb.vtable_ = h->vt;
}
}

template <typename F>
void InitRepeatingCallbackFromFunctor(RepeatingCallback &cb, F &&functor) {
    using Fn = std::decay_t<F>;
    if constexpr (is_sbo_eligible_v<Fn, REPEATING_SBO_SIZE, REPEATING_SBO_ALIGN>) {
        cb.inline_vtable_.invoke = [](char *storage) { std::invoke(*reinterpret_cast<Fn *>
(storage)); };
        cb.inline_vtable_.copy_construct = [](char *dst, const char *src) {
            new (dst) Fn(*reinterpret_cast<const Fn *>(src));
        };
        cb.inline_vtable_.destroy = [](char *storage) { reinterpret_cast<Fn *>(storage)->~Fn();
};
        new (cb.inline_storage_) Fn(std::forward<F>(functor));
        cb.ctrl_ = nullptr;
    } else {
        struct Storage {
            RepeatingControlBlock ctrl;
            Fn fn;
        };

        auto *s = static_cast<Storage *>(callback_alloc(sizeof(Storage)));
        s->ctrl.invoke = [](RepeatingControlBlock *self) { std::invoke(reinterpret_cast<Storage
*>(self)->fn); };
        s->ctrl.destroy = [](RepeatingControlBlock *self) {
            if (self->ref_count.fetch_sub(1, std::memory_order_acq_rel) == 1) {
                reinterpret_cast<Storage *>(self)->fn.~Fn();
                callback_free(self);
            }
        };
        new (&s->ctrl.ref_count) std::atomic<int>(1);
        new (&s->fn) Fn(std::forward<F>(functor));
        cb.inline_vtable_ = {nullptr, nullptr, nullptr};
        cb.ctrl_ = &s->ctrl;
    }
}

} // namespace detail

} // namespace nei

```

```
// --- Legacy aliases (kept for transition) -----
// OnceClosure / RepeatingClosure are defined in task_runner.h as typedefs.

#endif // NEI_TASK_CALLBACK_H
```

neixx/functional/include/neixx/functional/callback_base.h

```
#pragma once

#ifndef NEIXX_FUNCTIONAL_CALLBACK_BASE_H_
#define NEIXX_FUNCTIONAL_CALLBACK_BASE_H_

#include <nei/macros/nei_export.h>

namespace nei {

// Shared abstraction for callback wrappers.
// This keeps the null-state query behind a stable virtual interface while
// leaving invocation semantics in the concrete callback types.
class NEI_API CallbackBase {
public:
    virtual ~CallbackBase();

    CallbackBase(const CallbackBase &) = default;
    CallbackBase &operator=(const CallbackBase &) = default;
    CallbackBase(CallbackBase &&) noexcept = default;
    CallbackBase &operator=(CallbackBase &&) noexcept = default;

    bool IsNull() const noexcept {
        return IsNullImpl();
    }

protected:
    CallbackBase() = default;

private:
    virtual bool IsNullImpl() const noexcept = 0;
};

} // namespace nei

#endif // NEIXX_FUNCTIONAL_CALLBACK_BASE_H_
```

neixx/functional/include/neixx/functional/callback_internal.h

```
#pragma once

#ifndef NEIXX_FUNCTIONAL_CALLBACK_INTERNAL_H_
#define NEIXX_FUNCTIONAL_CALLBACK_INTERNAL_H_

#include <cstddef>
```

```

#include <new>
#include <type_traits>
#include <utility>

#include <neixx/memory/unretained_wrapper.h>

namespace nei {

// Forward declaration - avoids pulling in the full weak_ptr.h header here.
// Callers that actually pass a WeakPtr to BindOnce/BindRepeating must include
// <neixx/memory/weak_ptr.h> themselves so that operator bool() is visible at the
// instantiation site.
template <typename T>
class WeakPtr;

// Forward declaration so that nei::detail can reference nei::PassedWrapper
// before the full class definition below.
template <typename T>
class PassedWrapper;

namespace detail {

// --- Allocation primitives -----
//
// All callback heap paths route through these two helpers.
// Replace the bodies (at startup, before any callbacks are created) to plug in
// a custom memory pool. Thread-safety during replacement is the caller's
// responsibility.
//
inline void *callback_alloc(std::size_t bytes) {
    return ::operator new(bytes);
}

inline void callback_free(void *ptr) noexcept {
    ::operator delete(ptr);
}

// --- SBO eligibility -----
//
// True when a type T can be stored inline in an SBO buffer of the given size
// and alignment.
template <typename T, std::size_t SboSize, std::size_t SboAlign>
constexpr bool is_sbo_eligible_v = sizeof(T) <= SboSize && alignof(T) <= SboAlign;

// --- WeakPtr detection -----
//
// Used by BindOnce / BindRepeating to detect when the first bound argument is
// a WeakPtr<T> so that an automatic validity check can be injected.
template <typename T>
struct is_weak_ptr : std::false_type {};

template <typename T>
struct is_weak_ptr<nei::WeakPtr<T>> : std::true_type {};

```

```

template <typename T>
constexpr bool is_weak_ptr_v = is_weak_ptr<T>::value;

template <typename T>
struct is_unretained_wrapper : std::false_type {};

template <typename T>
struct is_unretained_wrapper<nei::UnretainedWrapper<T>> : std::true_type {};

template <typename T>
constexpr bool is_unretained_wrapper_v = is_unretained_wrapper<T>::value;

// --- PassedWrapper detection -----

template <typename T>
struct is_passed_wrapper : std::false_type {};

template <typename T>
struct is_passed_wrapper<nei::PassedWrapper<T>> : std::true_type {};

template <typename T>
constexpr bool is_passed_wrapper_v = is_passed_wrapper<T>::value;

template <typename T>
struct bind_arg_storage_impl {
    using type = std::decay_t<T>;

    template <typename U>
    static type Store(U &&value) {
        return std::forward<U>(value);
    }
};

template <typename T>
struct bind_arg_storage_impl<nei::UnretainedWrapper<T>> {
    using type = T *;

    static type Store(nei::UnretainedWrapper<T> value) {
        return value.get();
    }
};

// PassedWrapper: unwrap immediately at bind time - store the held object
// directly as std::decay_t<T>. At invoke time the regular UnwrapBoundArg
// overload moves it out as an rvalue, satisfying one-shot move semantics.
template <typename T>
struct bind_arg_storage_impl<nei::PassedWrapper<T>> {
    using type = std::decay_t<T>;

    static type Store(nei::PassedWrapper<T> value) {
        return value.Take();
    }
}

```

```

};

template <typename T>
struct bind_arg_storage :
bind_arg_storage_impl<std::remove_cv_t<std::remove_reference_t<T>>> {}>> {};

template <typename T>
using bind_arg_storage_t = typename
bind_arg_storage<std::remove_cv_t<std::remove_reference_t<T>>>::type;

template <typename T>
bind_arg_storage_t<T> StoreBoundArg(T &&value) {
    return bind_arg_storage<T>::Store(std::forward<T>(value));
}

// --- UnwrapBoundArg -----
//
// Called at invoke time to extract the argument from its stored form.
// All bound args (including those that originated from Passed()) are stored
// as plain values, so a single unconditional move covers every case.

template <typename T>
T &&UnwrapBoundArg(T &arg) noexcept {
    return std::move(arg);
}

} // namespace detail

// --- PassedWrapper -----
//
// Wraps a move-only object so that BindOnce can accept and forward it without
// copying. The object is moved into the wrapper at bind time and moved out
// of it exactly once when the resulting OnceCallback is invoked.
//
// Usage:
//   auto ptr = std::make_unique<Foo>();
//   auto cb = BindOnce(&HandleFoo, Passed(std::move(ptr)));
//
template <typename T>
class PassedWrapper {
public:
    explicit PassedWrapper(T &&t) : value_(std::move(t)) {}

    PassedWrapper(const PassedWrapper &) = delete;
    PassedWrapper &operator=(const PassedWrapper &) = delete;
    PassedWrapper(PassedWrapper &&) = default;
    PassedWrapper &operator=(PassedWrapper &&) = default;

    // Extract the held value. Must only be called once.
    T &&Take() { return std::move(value_); }

private:
    T value_;

```

```
};

// Convenience factory - mirrors Chromium's base::Passed().
template <typename T>
PassedWrapper<std::decay_t<T>> Passed(T &&t) {
    return PassedWrapper<std::decay_t<T>>(std::forward<T>(t));
}

} // namespace nei

#endif // NEIXX_FUNCTIONAL_CALLBACK_INTERNAL_H_
```

neixx/functional/include/neixx/functional/cancelable_callback.h

```
#pragma once

#ifndef NEI_FUNCTIONAL_CANCELABLE_CALLBACK_H
#define NEI_FUNCTIONAL_CANCELABLE_CALLBACK_H

#include <memory>

#include <nei/macros/nei_export.h>

namespace nei {

class OnceCallback;

#ifdef _MSC_VER
#pragma warning(push)
#pragma warning(disable : 4251)
#endif

class NEI_API CancelableOnceClosure final {
public:
    class Impl;

    explicit CancelableOnceClosure(OnceCallback closure);
    ~CancelableOnceClosure();

    CancelableOnceClosure(const CancelableOnceClosure &) = delete;
    CancelableOnceClosure &operator=(const CancelableOnceClosure &) = delete;

    CancelableOnceClosure(CancelableOnceClosure &&other) noexcept;
    CancelableOnceClosure &operator=(CancelableOnceClosure &&other) noexcept;

    OnceCallback callback();
    void Cancel();

private:
    std::unique_ptr<Impl> impl_;
};
```

```

#ifdef _MSC_VER
#pragma warning(pop)
#endif

} // namespace nei

#endif // NEI_FUNCTIONAL_CANCELABLE_CALLBACK_H

```

neixx/functional/src/callback.cpp

```

// callback.cpp - out-of-line definitions for OnceCallback and RepeatingCallback.
//
// All non-template lifecycle methods are compiled once into nei.dll.
// Template factory helpers that materialize concrete functors are defined in
// callback.h within nei::detail, keeping class bodies non-template oriented
// while preserving per-TU template instantiation visibility.

#include <neixx/functional/callback.h>

#include <cstring>
#include <utility>

namespace nei {

// --- OnceCallback -----

OnceCallback::OnceCallback() noexcept
    : vtable_{nullptr, nullptr} {
    std::memset(storage_, 0, detail::ONCE_SBO_SIZE);
}

OnceCallback::~~OnceCallback() {
    if (vtable_.destroy) {
        vtable_.destroy(storage_);
    }
}

OnceCallback::OnceCallback(OnceCallback &&other) noexcept
    : vtable_(other.vtable_) {
    std::memcpy(storage_, other.storage_, detail::ONCE_SBO_SIZE);
    // Clear the source
    other.vtable_ = {nullptr, nullptr};
    std::memset(other.storage_, 0, detail::ONCE_SBO_SIZE);
}

OnceCallback &OnceCallback::operator=(OnceCallback &&other) noexcept {
    if (this != &other) {
        // Destroy current
        if (vtable_.destroy) {
            vtable_.destroy(storage_);
        }
        // Move from other

```

```

    vtable_ = other.vtable_;
    std::memcpy(storage_, other.storage_, detail::ONCE_SBO_SIZE);
    // Clear the source
    other.vtable_ = {nullptr, nullptr};
    std::memset(other.storage_, 0, detail::ONCE_SBO_SIZE);
}
return *this;
}

OnceCallback::operator bool() const noexcept {
    return !IsNull();
}

void OnceCallback::Run() && {
    if (vtable_.invoke_and_destroy) {
        vtable_.invoke_and_destroy(storage_);
        // vtable was consumed by invoke_and_destroy, clear it
        vtable_ = {nullptr, nullptr};
        std::memset(storage_, 0, detail::ONCE_SBO_SIZE);
    }
}

// --- RepeatingCallback -----

RepeatingCallback::RepeatingCallback() noexcept
    : inline_vtable_{nullptr, nullptr, nullptr}
    , ctrl_(nullptr) {}

RepeatingCallback::RepeatingCallback(detail::RepeatingControlBlock *ctrl) noexcept
    : inline_vtable_{nullptr, nullptr, nullptr}
    , ctrl_(ctrl) {}

RepeatingCallback::RepeatingCallback(const RepeatingCallback &other) noexcept
    : inline_vtable_(other.inline_vtable_)
    , ctrl_(nullptr) {
    if (inline_vtable_.invoke) {
        // Inline path: copy-construct functor into own storage.
        // noexcept is maintained under the assumption that inline functors
        // (small lambdas that fit in SBO) have noexcept copy constructors.
        inline_vtable_.copy_construct(inline_storage_, other.inline_storage_);
    } else if (other.ctrl_) {
        // Heap path: share the ref-counted control block.
        ctrl_ = other.ctrl_;
        ctrl_>ref_count.fetch_add(1, std::memory_order_relaxed);
    }
}

RepeatingCallback &RepeatingCallback::operator=(const RepeatingCallback &other) noexcept {
    if (this != &other) {
        // Destroy current state.
        if (inline_vtable_.destroy) {

```



```

        inline_vtable_.destroy(inline_storage_);
    } else if (ctrl_) {
        ctrl_>destroy(ctrl_);
    }
    // Copy from other.
    inline_vtable_ = other.inline_vtable_;
    ctrl_ = nullptr;
    if (inline_vtable_.invoke) {
        inline_vtable_.copy_construct(inline_storage_, other.inline_storage_);
    } else if (other.ctrl_) {
        ctrl_ = other.ctrl_;
        ctrl_>ref_count.fetch_add(1, std::memory_order_relaxed);
    }
}
return *this;
}

```

```

RepeatingCallback::RepeatingCallback(RepeatingCallback &&other) noexcept
    : inline_vtable_(other.inline_vtable_)
    , ctrl_(nullptr) {
    if (inline_vtable_.invoke) {
        // Inline path: copy-construct then destroy source.
        // Since RepeatingCallback is copyable, copy+destroy is semantically a move.
        inline_vtable_.copy_construct(inline_storage_, other.inline_storage_);
        inline_vtable_.destroy(other.inline_storage_);
        other.inline_vtable_ = {nullptr, nullptr, nullptr};
    } else {
        // Heap path: transfer the control block pointer.
        ctrl_ = other.ctrl_;
        other.ctrl_ = nullptr;
    }
}

```

```

RepeatingCallback &RepeatingCallback::operator=(RepeatingCallback &&other) noexcept {
    if (this != &other) {
        // Destroy current state.
        if (inline_vtable_.destroy) {
            inline_vtable_.destroy(inline_storage_);
        } else if (ctrl_) {
            ctrl_>destroy(ctrl_);
        }
        // Move from other.
        inline_vtable_ = other.inline_vtable_;
        ctrl_ = nullptr;
        if (inline_vtable_.invoke) {
            inline_vtable_.copy_construct(inline_storage_, other.inline_storage_);
            inline_vtable_.destroy(other.inline_storage_);
            other.inline_vtable_ = {nullptr, nullptr, nullptr};
        } else {
            ctrl_ = other.ctrl_;
            other.ctrl_ = nullptr;
        }
    }
}

```

```

    return *this;
}

RepeatingCallback::~RepeatingCallback() {
    if (inline_vtable_.destroy) {
        inline_vtable_.destroy(inline_storage_);
    } else if (ctrl_) {
        ctrl_>destroy(ctrl_);
    }
}

RepeatingCallback::operator bool() const noexcept {
    return !IsNull();
}

void RepeatingCallback::Run() const {
    if (inline_vtable_.invoke) {
        inline_vtable_.invoke(inline_storage_);
    } else if (ctrl_) {
        ctrl_>invoke(ctrl_);
    }
}

} // namespace nei

```

neixx/functional/src/callback_base.cpp

```

#include <neixx/functional/callback_base.h>

namespace nei {

CallbackBase::~CallbackBase() = default;

} // namespace nei

```

neixx/functional/src/cancelable_callback.cpp

```

#include <neixx/functional/cancelable_callback.h>
#include <neixx/functional/bind.h>
#include <neixx/memory/weak_ptr.h>

#include <atomic>
#include <mutex>
#include <utility>

namespace nei {

class CancelableOnceClosure::Impl {
public:
    explicit Impl(OnceCallback closure)
        : state_(std::move(closure)) {

```

```

}

OnceCallback callback() {
    weakPtr<State> weak_state = state_.GetWeakPtr();
    return BindOnce([weak_state]() mutable {
        if (!weak_state) {
            return;
        }

        if (weak_state->is_cancelled.load(std::memory_order_acquire)) {
            return;
        }

        OnceCallback task;
        {
            std::lock_guard<std::mutex> lock(weak_state->mutex);
            if (weak_state->is_cancelled.load(std::memory_order_relaxed)) {
                return;
            }
            task = std::move(weak_state->task);
        }

        if (!task) {
            return;
        }

        if (weak_state->is_cancelled.load(std::memory_order_acquire)) {
            return;
        }

        std::move(task).Run();
    });
}

void Cancel() {
    state_.Cancel();
}

private:
    struct State {
        explicit State(OnceCallback closure_in)
            : task(std::move(closure_in))
            , weak_factory(this) {
        }

        weakPtr<State> GetWeakPtr() {
            return weak_factory.GetWeakPtr();
        }

        void Cancel() {
            is_cancelled.store(true, std::memory_order_release);
            std::lock_guard<std::mutex> lock(mutex);
            task = OnceCallback();
        }
    };

```

```

        weak_factory.InvalidateWeakPtrs();
    }

    std::atomic<bool> is_cancelled{false};
    std::mutex mutex;
    OnceCallback task;
    WeakPtrFactory<State> weak_factory;
};

State state_;
};

CancelableOnceClosure::CancelableOnceClosure(OnceCallback closure)
    : impl_(std::make_unique<Impl>(std::move(closure))) {}

CancelableOnceClosure::~CancelableOnceClosure() {
    Cancel();
}

CancelableOnceClosure::CancelableOnceClosure(CancelableOnceClosure &&other) noexcept =
default;

CancelableOnceClosure &CancelableOnceClosure::operator=(CancelableOnceClosure &&other)
noexcept {
    if (this != &other) {
        Cancel();
        impl_ = std::move(other.impl_);
    }
    return *this;
}

OnceCallback CancelableOnceClosure::callback() {
    if (!impl_) {
        return OnceCallback();
    }
    return impl_>callback();
}

void CancelableOnceClosure::Cancel() {
    if (!impl_) {
        return;
    }
    impl_>Cancel();
}

} // namespace nei

```

neixx/memory/include/neixx/memory/internal_flag.h

```
#pragma once

#ifndef NEIXX_MEMORY_INTERNAL_FLAG_H_
#define NEIXX_MEMORY_INTERNAL_FLAG_H_

#include <memory>

#include <nei/macros/nei_export.h>

namespace nei {

#ifdef _MSC_VER
#pragma warning(push)
#pragma warning(disable : 4251)
#endif

// Shared validity flag used by WeakPtrFactory/WeakPtr.
// Invalidated (once) when the factory is destroyed.
class NEI_API InternalFlag final {
public:
    class Impl;

    InternalFlag();
    ~InternalFlag();

    InternalFlag(const InternalFlag &) = delete;
    InternalFlag &operator=(const InternalFlag &) = delete;

    bool IsValid() const;
    void Invalidate();

private:
    std::unique_ptr<Impl> impl_;
};

#ifdef _MSC_VER
#pragma warning(pop)
#endif

} // namespace nei

#endif // NEIXX_MEMORY_INTERNAL_FLAG_H_
```

neixx/memory/include/neixx/memory/pass_key.h

```
#ifndef NEIXX_MEMORY_PASS_KEY_H_
#define NEIXX_MEMORY_PASS_KEY_H_

#include <nei/macros/nei_export.h>
```

```

namespace nei {

template <typename T>
class NEI_API PassKey final {
private:
    friend T;
    PassKey() = default;
};

} // namespace nei

#endif // NEIXX_MEMORY_PASS_KEY_H_

```

neixx/memory/include/neixx/memory/ref_counted.h

```

#ifndef NEIXX_MEMORY_REF_COUNTED_H_
#define NEIXX_MEMORY_REF_COUNTED_H_

#include <type_traits>
#include <utility>

#include <nei/macros/nei_export.h>

#if defined(_MSC_VER)
#include <intrin.h>
#else
#include <atomic>
#endif

namespace nei {

namespace detail {

template <typename T, typename = void>
struct IsRefCountedLike : std::false_type {
};

template <typename T>
struct IsRefCountedLike<T,
    std::void_t<decltype(std::declval<const T &>().AddRef()),
                decltype(std::declval<const T &>().Release())>>
    : std::bool_constant<std::is_same_v<decltype(std::declval<const T &>().AddRef()),
                                        void> &&
                        std::is_same_v<decltype(std::declval<const T &>().Release()),
                                        void>>> {
};

#if defined(__cpp_concepts) && __cpp_concepts >= 201907L
template <typename T>
concept RefCountedLike = IsRefCountedLike<T>::value;
#endif

}

```

```

} // namespace detail

// Non-template exported base that holds the reference counter.
// By exporting this concrete class (NEI_API), the DLL provides the reference-
// count storage for all RefCountedThreadSafe<T> subclasses. The member type
// is a platform primitive (volatile long on MSVC, std::atomic<int> elsewhere)
// so MSVC warning C4251 is not triggered on windows.
class NEI_API RefCountBase {
    template <typename T>
    friend class RefCountedThreadSafe;

protected:
    RefCountBase() noexcept = default;
    ~RefCountBase() = default;

#ifdef _MSC_VER
    void AddRefImpl() const noexcept { _InterlockedIncrement(&ref_count_); }
    bool ReleaseImpl() const noexcept { return _InterlockedDecrement(&ref_count_) == 0; }
private:
    mutable volatile long ref_count_ = 0;
#else
    void AddRefImpl() const noexcept { ref_count_.fetch_add(1, std::memory_order_relaxed); }
    bool ReleaseImpl() const noexcept { return ref_count_.fetch_sub(1,
std::memory_order_acq_rel) == 1; }
private:
    mutable std::atomic<int> ref_count_{0};
#endif
};

// Intrusive, thread-safe reference counting base class.
//
// Usage:
// 1) Derive `T` from `RefCountedThreadSafe<T>`.
// 2) Manage lifetime through `scoped_refptr<T>`.
//
// The reference counter starts at 0. A `scoped_refptr` created from a raw
// pointer will call `AddRef()`. When the last reference releases, `T` is
// deleted on the releasing thread.
template <typename T>
class RefCountedThreadSafe : private RefCountBase {
public:
    // Increments the reference count.
    // Relaxed ordering is sufficient because this operation only updates the
    // counter value and does not publish object state.
    void AddRef() const noexcept {
        AddRefImpl();
    }

    // Decrements the reference count and destroys the object at zero.
    // Acquire-release ordering pairs with prior writes performed by owners and
    // ensures destruction observes a fully initialized object state.
    void Release() const noexcept {

```

```

    if (ReleaseImpl()) {
        delete static_cast<const T *>(this);
    }
}

protected:
    RefCountedThreadSafe() noexcept = default;
    ~RefCountedThreadSafe() = default;
};

// A lightweight intrusive smart pointer for `RefCountedThreadSafe` objects.
//
// `scoped_refptr` owns one reference while holding a non-null pointer and
// automatically balances `AddRef()` / `Release()` across copy/move/reset.
#if defined(__cpp_concepts) && __cpp_concepts >= 201907L
template <detail::RefCountedLike T>
#else
template <typename T>
#endif
class scoped_refptr {
public:
#if !defined(__cpp_concepts) || __cpp_concepts < 201907L
    static_assert(detail::IsRefCountedLike<T>::value,
        "scoped_refptr<T> requires T to provide const AddRef() and const "
        "Release() returning void");
#endif

    scoped_refptr() noexcept = default;

    scoped_refptr(std::nullptr_t) noexcept : ptr_(nullptr) {}

    // Takes a raw pointer and acquires one reference if non-null.
    explicit scoped_refptr(T *ptr) noexcept : ptr_(ptr) {
        AddRefIfNeeded();
    }

    scoped_refptr(const scoped_refptr &other) noexcept : ptr_(other.ptr_) {
        AddRefIfNeeded();
    }

    scoped_refptr(scoped_refptr &&other) noexcept : ptr_(other.ptr_) {
        other.ptr_ = nullptr;
    }

    ~scoped_refptr() {
        ReleaseIfNeeded();
    }

    scoped_refptr &operator=(const scoped_refptr &other) noexcept {
        if (this != &other) {
            scoped_refptr tmp(other);
            swap(tmp);
        }
    }

```



```

    }
    return *this;
}

scoped_refptr &operator=(scoped_refptr &&other) noexcept {
    if (this != &other) {
        ReleaseIfNeeded();
        ptr_ = other.ptr_;
        other.ptr_ = nullptr;
    }
    return *this;
}

scoped_refptr &operator=(std::nullptr_t) noexcept {
    ReleaseIfNeeded();
    ptr_ = nullptr;
    return *this;
}

T *get() const noexcept {
    return ptr_;
}

T &operator*() const noexcept {
    return *ptr_;
}

T *operator->() const noexcept {
    return ptr_;
}

explicit operator bool() const noexcept {
    return ptr_ != nullptr;
}

// Replaces the owned pointer while preserving strong exception safety.
void reset(T *ptr = nullptr) noexcept {
    if (ptr_ != ptr) {
        scoped_refptr tmp(ptr);
        swap(tmp);
    }
}

void swap(scoped_refptr &other) noexcept {
    std::swap(ptr_, other.ptr_);
}

private:
void AddRefIfNeeded() noexcept {
    if (ptr_ != nullptr) {
        ptr_->AddRef();
    }
}

```

```

void ReleaseIfNeeded() noexcept {
    if (ptr_ != nullptr) {
        ptr_>Release();
        ptr_ = nullptr;
    }
}

T *ptr_ = nullptr;
};

// Factory helper that creates a ref-counted object and returns it as
// `scoped_refptr<T>`.
#if defined(__cpp_concepts) && __cpp_concepts >= 201907L
template <detail::RefCountedLike T, typename... Args>
#else
template <typename T, typename... Args>
#endif
scoped_refptr<T> MakeRefCounted(Args &&...args) {
    if !defined(__cpp_concepts) || __cpp_concepts < 201907L
        static_assert(detail::IsRefCountedLike<T>::value,
            "MakeRefCounted<T> requires T to provide const AddRef() and const "
            "Release() returning void");
    #endif
    return scoped_refptr<T>(new T(std::forward<Args>(args)...));
}

} // namespace nei

#endif // NEIXX_MEMORY_REF_COUNTED_H_

```

neixx/memory/include/neixx/memory/unretained_wrapper.h

```

#ifndef NEIXX_MEMORY_UNRETAINED_WRAPPER_H_
#define NEIXX_MEMORY_UNRETAINED_WRAPPER_H_

#include <nei/macros/nei_export.h>

namespace nei {

template <typename T>
class NEI_API UnretainedWrapper final {
public:
    explicit UnretainedWrapper(T *ptr) : ptr_(ptr) {
    }

    T *get() const {
        return ptr_;
    }

private:
    T *ptr_ = nullptr;
};

```

```
};

template <typename T>
inline UnretainedWrapper<T> Unretained(T *ptr) {
    return UnretainedWrapper<T>(ptr);
}

} // namespace nei

#endif // NEIXX_MEMORY_UNRETAINED_WRAPPER_H_
```

neixx/memory/include/neixx/memory/weak_ptr.h

```
#pragma once

#ifndef NEI_TASK_WEAK_PTR_H
#define NEI_TASK_WEAK_PTR_H

#include <cassert>
#include <memory>
#include <thread>
#include <type_traits>

#include <neixx/memory/internal_flag.h>

namespace nei {

template <typename T>
struct WeakPtrThreadSafe : std::false_type {};

template <typename T>
class WeakPtr;

template <typename T>
class WeakPtrFactory {
public:
    explicit WeakPtrFactory(T *ptr)
        : ptr_(ptr)
        , flag_(std::make_shared<InternalFlag>())
        , bound_thread_(std::this_thread::get_id()) {
    }

    ~WeakPtrFactory() {
        InvalidateWeakPtrs();
    }

    WeakPtrFactory(const WeakPtrFactory &) = delete;
    WeakPtrFactory &operator=(const WeakPtrFactory &) = delete;

    WeakPtr<T> GetWeakPtr() const {
        return WeakPtr<T>(ptr_, flag_, bound_thread_);
    }
};
```

```

void InvalidateWeakPtrs() {
    if (!flag_) {
        return;
    }
    flag_->Invalidate();
    flag_.reset();
}

private:
    T *ptr_;
    std::shared_ptr<InternalFlag> flag_;
    std::thread::id bound_thread_;
};

template <typename T>
class WeakPtr {
public:
    WeakPtr() = default;

    T *get() const {
#ifdef NDEBUG
        AssertThreadSafeDereference();
#endif
        return IsValid() ? ptr_ : nullptr;
    }

    T *operator->() const {
        return get();
    }

    T &operator*() const {
        return *get();
    }

    explicit operator bool() const {
        return IsValid();
    }

private:
    friend class WeakPtrFactory<T>;

    WeakPtr(T *ptr, const std::shared_ptr<InternalFlag> &flag, std::thread::id bound_thread)
        : ptr_(ptr)
        , flag_(flag)
        , bound_thread_(std::move(bound_thread)) {}

    bool IsValid() const {
        std::shared_ptr<InternalFlag> flag = flag_.lock();
        return flag && flag->IsValid();
    }
}

```

```

#if !defined(NDEBUG)
    void AssertThreadSafeDereference() const {
        if (ptr_ == nullptr || weakPtrThreadSafe<T>::value) {
            return;
        }
        assert(bound_thread_ == std::this_thread::get_id()
            && "WeakPtr dereferenced across threads without weakPtrThreadSafe opt-in");
    }
#endif

    T *ptr_ = nullptr;
    std::weak_ptr<InternalFlag> flag_;
    std::thread::id bound_thread_{};
};

} // namespace nei

#endif // NEI_TASK_WEAK_PTR_H

```

neixx/memory/src/internal_flag.cpp

```

#include <neixx/memory/internal_flag.h>

#include <atomic>
#include <memory>

namespace nei {

class InternalFlag::Impl {
public:
    Impl()
        : valid_(std::make_shared<std::atomic<bool>>(true)) {
    }

    bool IsValid() const {
        return valid_>load(std::memory_order_acquire);
    }

    void Invalidate() {
        valid_>store(false, std::memory_order_release);
    }

private:
    std::shared_ptr<std::atomic<bool>> valid_;
};

InternalFlag::InternalFlag()
    : impl_(std::make_unique<Impl>()) {
}

InternalFlag::~InternalFlag() {
    impl_>Invalidate();
}

```

```

}

bool InternalFlag::IsValid() const {
    return impl_>IsValid();
}

void InternalFlag::Invalidate() {
    impl_>Invalidate();
}

} // namespace nei

```

neixx/synchronization/include/synchronization/waitable_event.h

```

#pragma once

#ifndef NEIXX_THREADING_WAITABLE_EVENT_H_
#define NEIXX_THREADING_WAITABLE_EVENT_H_

#include <chrono>
#include <memory>

#include <nei/macros/nei_export.h>

namespace nei {

class NEI_API WaitableEvent final {
public:
    enum class ResetPolicy {
        kManual,
        kAutomatic,
    };

    class Impl;

    explicit WaitableEvent(ResetPolicy reset_policy, bool initially_signaled = false);
    ~WaitableEvent();

    WaitableEvent(const WaitableEvent &) = delete;
    WaitableEvent &operator=(const WaitableEvent &) = delete;
    WaitableEvent(WaitableEvent &&) noexcept;
    WaitableEvent &operator=(WaitableEvent &&) noexcept;

    void Signal();
    void Wait();
    bool Timedwait(std::chrono::milliseconds timeout);

private:
    std::unique_ptr<Impl> impl_;
};

} // namespace nei

```

```
#endif // NEIXX_THREADING_WAITABLE_EVENT_H_
```

neixx/synchronization/src/waitable_event.cpp

```
#include <neixx/threading/waitable_event.h>

#include <neixx/threading/thread_restrictions.h>

#ifdef _WIN32
#include <windows.h>
#endif

#include <condition_variable>
#include <mutex>

namespace nei {

class WaitableEvent::Impl {
public:
    Impl(ResetPolicy reset_policy, bool initially_signaled)
        : reset_policy_(reset_policy)
#ifdef _WIN32
        , event_handle_(CreateEventA(nullptr,
                                     reset_policy == ResetPolicy::kManual ? TRUE : FALSE,
                                     initially_signaled ? TRUE : FALSE,
                                     nullptr))
#else
        , signaled_(initially_signaled)
#endif
    {
    }

    ~Impl() {
#ifdef _WIN32
        if (event_handle_ != nullptr) {
            CloseHandle(event_handle_);
            event_handle_ = nullptr;
        }
#endif
    }

    void Signal() {
#ifdef _WIN32
        if (event_handle_ != nullptr) {
            SetEvent(event_handle_);
        }
#else
        {
            std::lock_guard<std::mutex> lock(mutex_);
            signaled_ = true;
        }
#endif
    }
};

} // namespace nei
```

```

    if (reset_policy_ == ResetPolicy::kManual) {
        cv_.notify_all();
    } else {
        cv_.notify_one();
    }
#endif
}

void Wait() {
#ifdef _WIN32
    if (event_handle_ != nullptr) {
        WaitForSingleObject(event_handle_, INFINITE);
    }
#else
    std::unique_lock<std::mutex> lock(mutex_);
    cv_.wait(lock, [this]() { return signaled_; });
    if (reset_policy_ == ResetPolicy::kAutomatic) {
        signaled_ = false;
    }
#endif
}

bool Timedwait(std::chrono::milliseconds timeout) {
#ifdef _WIN32
    if (event_handle_ == nullptr) {
        return false;
    }
    const DWORD wait_result = WaitForSingleObject(event_handle_, static_cast<DWORD>
(timeout.count()));
    return wait_result == WAIT_OBJECT_0;
#else
    std::unique_lock<std::mutex> lock(mutex_);
    const bool signaled = cv_.wait_for(lock, timeout, [this]() { return signaled_; });
    if (!signaled) {
        return false;
    }
    if (reset_policy_ == ResetPolicy::kAutomatic) {
        signaled_ = false;
    }
    return true;
#endif
}

private:
    ResetPolicy reset_policy_;

#ifdef _WIN32
    HANDLE event_handle_ = nullptr;
#else
    std::mutex mutex_;
    std::condition_variable cv_;
    bool signaled_ = false;
#endif
}

```



```

};

waitableEvent::waitableEvent(ResetPolicy reset_policy, bool initially_signaled)
    : impl_(std::make_unique<Impl>(reset_policy, initially_signaled)) {
}

waitableEvent::~~waitableEvent() = default;

waitableEvent::waitableEvent(waitableEvent &&) noexcept = default;

waitableEvent &waitableEvent::operator=(waitableEvent &&) noexcept = default;

void waitableEvent::Signal() {
    impl_->Signal();
}

void waitableEvent::Wait() {
    ThreadRestrictions::AssertBaseSyncPrimitivesAllowed();
    impl_->wait();
}

bool waitableEvent::TimedWait(std::chrono::milliseconds timeout) {
    return impl_->Timedwait(timeout);
}

} // namespace nei

```